Singapore Housing Macro Data Agent Engineering Report

This report describes the engineering design of an agent-enabled workflow for producing reproducible analysis of publicly available indicators relevant to Singapore residential sale and rental markets. The workflow separates flexible research and interpretation from deterministic data processing so that every reported value remains reviewable, reproducible and traceable to preserved source evidence.

1. System Design

The workflow is implemented in Python and orchestrated with LangGraph. Pydantic models define structured contracts between stages, while OpenPyXL and standard JSON, CSV and HTML utilities support the public-data formats used in the workflow. LangGraph is used because the process includes explicit approval and recovery points rather than following a purely linear script: research or source-mapping outputs may fail validation, sources may need replacement, and report drafts may require revision. Numerical processing, however, is kept outside the LLM and performed by deterministic Python.

```
User Scope -> Stage 1 Agent Research -> Human Predictor Approval
    -> Stage 2 Data Onboarding -> Human Data and Preparation Approval
    -> FrozenDataPlan -> Stage 3 Deterministic Processing
    -> ValidatedEvidenceBundle -> Stage 4 Report Writing and Verification
    -> Verified Final Report
```

The four stages deliberately separate flexible reasoning from numerical execution. In **Stage 1**, an LLM-assisted research agent interprets the confirmed scope, identifies literature-supported predictor concepts and explains their potential relevance to sale prices and rents. Deterministic checks validate the research output, and a human approves the final predictor set.

In **Stage 2**, agents assist with official-source discovery, unfamiliar-schema interpretation and preparation recommendations. Proposed sources and mappings are independently tested by code, while raw responses are preserved for reproducibility. A human approves the source, mapping, frequency treatment, change calculation and historical window before these decisions are frozen into a versioned `FrozenDataPlan`. Agent recommendations therefore cannot directly alter production processing.

Stage 3 is fully deterministic. Python verifies the approved plan and raw snapshots, normalises heterogeneous source formats, applies approved transformations, validates periods and units, and calculates changes such as QoQ, YoY, percentage-point and basis-point movements. The resulting observations and their provenance are assembled into a `ValidatedEvidenceBundle`. Unresolved validation failures block downstream report generation.

In **Stage 4**, an optional LLM writer uses only the validated evidence to organise the analysis and explain market mechanisms. It does not calculate or modify numerical values. Reported claims are linked to structured evidence and checked deterministically for their value, unit, period and source before a report is accepted. The full workflow therefore supports LLM-assisted research, onboarding and writing, while the committed sample can also be regenerated offline from the approved `FrozenDataPlan` and raw snapshots without requiring an API key.

Raw snapshots are stored separately from processed artifacts. JSON records preserve source metadata, retrieval information and checksums; normalised observations are stored as JSON and CSV; validation, lineage and evidence records are stored as JSON; and final reports are generated in Markdown. This file-based approach is portable and transparent for a small reproducible workflow, although a database would be more appropriate at substantially larger scale.

Validation is applied throughout the pipeline rather than only to the final output. Typed schemas constrain agent outputs, deterministic checks verify sources and data quality, and the final verifier checks generated claims against the evidence used to produce them. This keeps the LLM behind a controlled evidence boundary rather than giving it unrestricted access to calculations or reported values.

The workflow is exposed through command-line interfaces because the task prioritises reproducibility over interface development. The full workflow provides separate commands for research, review, onboarding and approval, while the submitted result can be reproduced offline with:

```
.venv/bin/python scripts/run_submission_pipeline.py \
  --output outputs/submission_run \
  --print-report
```

The main design trade-off is **flexibility versus reliability**. LLMs are useful for semantic tasks such as research, source interpretation and narrative organisation, but their outputs are probabilistic. Deterministic components are less flexible but are reproducible, testable and auditable. The workflow therefore uses **agents to propose and explain decisions, humans to approve material choices, and deterministic code to control every reported number**.

2. Demonstration Case and Predictor Selection

2.1 Demonstration Scope

Singapore's residential market contains structurally different segments that may respond to different market and policy conditions. This demonstration therefore focuses on **private non-landed residential property**, covering both sale and rental markets as of **1 October 2026**. The objective is to identify informative public predictors for these two markets rather than train a forecasting model.

2.2 Agent Research and Human Selection

Given this scope, the Stage 1 research agent searches academic and official research for candidate macroeconomic, financial, demographic and housing-market factors. Each candidate is supported by an expected market mechanism, evidence and key limitations. After deterministic validation and human review, only approved predictors proceed to data onboarding. The complete pre-review agent output is provided in **Appendix A**.

2.3 Selected Predictors and Rationale

The final predictor set captures distinct mechanisms across the sale and rental markets. For sales, **3-month Compounded SORA** captures mortgage financing conditions, while **real GDP YoY growth** reflects broader economic conditions and **real GDP SA QoQ growth** captures shorter-term momentum. **Pipeline supply** is relevant to both markets by indicating prospective additions to housing stock.

For rentals, **vacancy rate** captures utilisation of existing stock. **Non-resident population** represents the stock of potential tenant demand, while **quarterly non-resident employment change** provides a more timely flow measure; they are retained separately because they capture different dimensions of rental demand.

The **URA private non-landed price and rental indices** are treated as outcome benchmarks rather than predictors. They provide the observed market movements against which these indicators are interpreted. These relationships are used as economically motivated monitoring hypotheses, not as estimated causal effects.

3. Data Acquisition, Transformation, Provenance and Reliability

3.1 Agent-assisted Data Onboarding and Approval

For each approved predictor and outcome, the Stage 2 agent proposes an official source and retrieval method. Code independently verifies the source and sampled structure before preserving the response as an immutable raw snapshot with retrieval metadata and a SHA-256 checksum. The agent may then propose a typed `MappingContract` and preparation rules, which are executed and validated against the full snapshot. A human approves the source, mapping, frequency treatment and change method before these decisions are frozen into a versioned `FrozenDataPlan`.

Re-running the same plan against the same snapshot reproduces the same artifacts; revised source data create a new snapshot, checksum and plan version rather than overwriting earlier evidence.

3.2 Raw Data Normalisation

Stage 3 first verifies the approved plan and snapshot checksums, then applies the `MappingContract` to convert heterogeneous JSON, HTML, CSV and Excel layouts into canonical observations. Each observation retains its series and source identifiers, period, decimal value, unit, observation and update frequencies, eligibility status, snapshot checksum and raw-record reference.

```
Raw:       "2026-Q2": "210.6"

Canonical: series_id = ura_non_landed_price_index
           period    = 2026-Q2
           value     = 210.6
           unit      = index; 2009-Q1 = 100
           raw_ref   = record:0/field:2026-Q2
```

Normalisation changes representation, not economic meaning. It may reshape records, parse approved periods and convert declared missing markers to null, but it does not impute, interpolate or calculate market changes.

3.3 Deterministic Transformations

Stage 3 makes no new semantic decisions. It executes only the transformations approved in the `FrozenDataPlan`, using configured Python functions that are frequency-, unit- and concept-aware. Quarterly URA price and rental indices use the exact preceding quarter for QoQ changes and the same quarter one year earlier for YoY changes. Pipeline supply remains a dwelling-unit count, while vacancy is derived from matched vacant and available units and changes are expressed in percentage points.

Monthly 3-month Compounded SORA uses the approved quarter-end observation only for completed quarters, with changes expressed in basis points. Official real GDP YoY and seasonally adjusted QoQ growth remain separate published measures. Non-resident population remains an annual stock without interpolation, while non-resident employment change remains a quarterly flow. These rules preserve each series' native frequency, unit and economic meaning rather than forcing all inputs into a common transformation.

3.4 Validation and Failure Handling

Validation runs before and after transformation, and the submission pipeline fails closed if the approved portfolio does not pass its required gates. Missing observations remain null; missing predecessors produce no change value; stale observations are flagged against release timing; and revised data create new snapshots and plan versions. Retrieval failures receive bounded retries; a required source that remains unavailable is routed to source review and blocks the validated portfolio rather than being silently replaced. Schema changes trigger mapping recovery, checksum mismatches hard-stop execution, and invalid arithmetic produces no derived observation.

Only a fully validated portfolio is assembled into the `ValidatedEvidenceBundle`.

3.5 Provenance and Validation Evidence

```
Official Source
    → Raw Snapshot + SHA-256
    → Raw Record Reference
    → Canonical Observation
    → Approved Transformation
    → Derived Observation
    → ValidatedEvidenceBundle
```

Every processed value retains its source and series identifiers, snapshot checksum, raw-record reference, input periods and applied derivation. A reviewer can therefore trace a derived value to the exact downloaded observations and independently reproduce the calculation.

Edge case. An early change calculation used the previous non-missing row, which could incorrectly compare 2026 Q3 with 2026 Q1 when Q2 was missing and label the result as QoQ. The implementation was changed to convert periods into calendar ordinals and require the exact predecessor. Regression tests now cover missing quarters, missing years and accidental cross-series comparisons. The submitted offline suite passes **186 tests**; a live-endpoint smoke test remains opt-in so that standard reproduction is deterministic and network-independent.

4. Evidence-Bound Reporting and Evaluation

Stage 4 converts the validated evidence into the final analytical report. It organises and interprets approved evidence while ensuring that every quantitative claim remains grounded in validated data and deterministic verification.

4.1 Evidence-Bound Report Generation

```
ValidatedEvidenceBundle -> Evidence Selector
    -> Writer: deterministic fallback or optional LLM
    -> Deterministic Composer -> Claim Verifier -> Verified Final Report
```

Either an optional LLM writer or a deterministic fallback organises and explains the qualitative evidence selected from the `ValidatedEvidenceBundle`. The writer decides how to structure the analysis and relate the selected indicators to the housing market, but it does not supply numerical values. A deterministic composer separately renders every approved value, unit, period and source from the validated evidence and records each quantitative statement in a claim ledger for verification.

The two writers follow the same evidence-bound structure but differ in how they generate the narrative. The optional LLM can flexibly synthesise different indicators, explain financing, demand, utilisation and supply mechanisms, and produce more natural transitions. The deterministic fallback uses predefined report logic and language, providing stable offline reproduction without an external model or API key. Both discuss possible time lags and avoid unsupported causal claims. The submitted sample uses the deterministic fallback.

4.2 Deterministic Claim Verification

The verifier checks that every reportable claim appears exactly once and matches its underlying evidence for value, unit, period and source. Numerical text not represented in the claim ledger is rejected, as are unsupported sources, missing required sections, causal claims and unqualified forecasts. The workflow permits at most three drafting attempts. If the final attempt still fails verification, it writes a blocked draft and machine-readable diagnostics rather than publishing a final report.

Representative verification tests:

```
tampered value · unledgered number · missing claim · duplicate claim
unsupported causal language · bounded drafting attempts
```

The submitted offline test suite passed **186 tests**, with one live-endpoint smoke test intentionally excluded from default execution to preserve network-independent reproduction. The end-to-end submission pipeline completed successfully, regeneration produced a byte-identical final report, and **19 report claims passed deterministic verification with 0 errors**.

4.3 Reviewer-facing Output

A successful run produces the final Markdown report together with the selected evidence, source register, prompt manifest, writer metadata, claim ledger, verification report and run manifest. These companion artifacts provide an auditable record of both the analytical output and the controls applied during its generation.

```
sample_run/submission/stage4/
├── final_report.md
├── selected_evidence.json
├── source_register.json
├── report_prompt_manifest.json
├── claim_ledger.json
├── verification_report.json
├── writer_metadata.json
└── run_manifest.json
```

5. Limitations, Next Steps and LLM Disclosure

5.1 Limitations

Report reproducibility depends on the writer used. Given identical inputs, the deterministic fallback produces a byte-identical report, whereas an LLM writer may vary in wording, emphasis and structure. This variation is constrained by a fixed evidence boundary, deterministic insertion of numerical values and claim verification. The system therefore keeps quantitative claims traceable and verifiable, but does not guarantee identical qualitative interpretation across LLM runs.

Although the workflow applies as-of-date and release-availability checks, it does not maintain a complete historical release-vintage archive for every source. Some official endpoints may expose revised historical observations rather than every version previously published. The workflow preserves the exact snapshot used in each run, but may not always reconstruct what was visible at an earlier historical date.

Indicators are released at different frequencies and times, so their latest available observations do not always refer to the same period. The workflow preserves these native frequencies and avoids artificial interpolation, but the timing differences still require care when interpreting relationships across indicators. The live Agent Flow also depends on external websites, official APIs and LLM providers. Retrieval, schema and mapping failures are detected and block unverified data, but material source changes may still require renewed research and human approval.

Repeated evaluation of the LLM-supported nodes was limited by API cost and provider quotas. The deterministic pipeline and offline fallback were tested extensively, but the submission does not evaluate LLM stability across large numbers of repeated calls, model versions or providers.

5.2 Next Two Weeks

The main extension would promote a successful, user-approved run into a persistent `ReusableWorkflowProfile`. With explicit approval, the profile would preserve the confirmed scope, predictor portfolio, source identities, mapping contracts, transformation rules, reporting preferences, report brief, writer prompt version, `FrozenDataPlan` checksum and latest successful validation result.

When a later request matches an existing profile, the system could offer to reuse the approved workflow while updating the as-of date and current data. This would avoid repeating predictor research, source selection and preparation discussions, reducing user interaction, execution time and LLM token usage. Fresh data would still be retrieved and validated. Any material change in scope, source, schema or series definition would reopen the relevant approval gate.

A second extension would strengthen source recovery by probing alternative official formats, such as CSV or Excel, when the preferred official API becomes unavailable or changes schema. The alternative representation would be checked for definition and schema compatibility before use. Any replacement that changes the source identity or series definition would require explicit user approval.

I would also conduct repeated cross-provider evaluations of the LLM-supported research, onboarding and reporting nodes. The evaluation would measure structured-output pass rates, retry frequency, latency, token cost, provider failures and consistency with approved decisions. For the report writer, it would additionally compare evidence coverage, treatment of mixed signals, preservation of limitations and writing quality against the deterministic fallback.

5.3 LLM and Agent Tool Disclosure

LangGraph orchestrates the workflow and maintains explicit state across research, onboarding, approval, processing and reporting. LLM-supported agents are used for predictor research, official-source discovery, schema interpretation, mapping and transformation recommendations, and optional qualitative report writing. OpenAI Codex was also used during development to support implementation review, testing and preparation of the engineering documentation.

Human approval is required for predictors, sources, mappings and preparation rules before they enter Project Memory and the `FrozenDataPlan`. LLMs do not perform authoritative numerical calculations, change approved Stage 3 rules or determine whether a report passes verification.

In Stage 4, the optional LLM writer produces only the qualitative narrative and does not supply quantitative values. All values, units, periods, lineage and source references are rendered separately from the `ValidatedEvidenceBundle` by deterministic code and checked by the claim verifier. The submitted sample uses the deterministic fallback, which generates a complete report without network access or an LLM API, while the full interactive Agent Flow and optional LLM writer remain available in the implementation.